Corrected Sentiment Transformer - Line by Line

Transformer Block Class

```
class TransformerBlock(nn.Module):
```

Standard transformer block with pre-normalization and correct mask handling.

```
    def __init__(self, embed_dim, num_heads, mlp_dim, dropout):
        super().__init__()
        # Pre-normalisation (meilleure stabilite que Post-LN)
        self.layer_norm_1 = nn.LayerNorm(embed_dim)
```

Creates first layer normalization for pre-LN architecture (normalizes before attention).

```
        self.attention = nn.MultiheadAttention(
            embed_dim,
            num_heads,
            dropout=dropout,
            batch_first=True
        )
```

Creates multi-head attention with `batch_first=True` for (batch, seq, features) format.

```
        self.layer_norm_2 = nn.LayerNorm(embed_dim)
```

Creates second layer normalization for MLP sublayer.

```
        # MLP avec GELU
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, mlp_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(mlp_dim, embed_dim),
            nn.Dropout(dropout)
        )
```

Creates MLP with GELU activation (smoother than ReLU).

```
    def forward(self, x, key_padding_mask=None):
```

Forward pass with optional padding mask. `key_padding_mask`: boolean tensor where True = mask this token.

```
        # === Attention avec residu (pre-normalisation) ===
        normed = self.layer_norm_1(x)
```

Normalizes input before attention (pre-LN style).

```
        # CORRECTION : utiliser key_padding_mask (pas attn_mask) pour le
            padding
        attn_out, _ = self.attention(
            normed, normed, normed,
```

```
            key_padding_mask=key_padding_mask,  # Format correct pour le
                padding
            attn_mask=None  # Pas de masque causal pour classification
        )
```

CRITICAL CORRECTION: Uses `key_padding_mask` for padding (not `attn_mask`). This prevents dimension errors with `nn.MultiheadAttention`.

```
        x = x + attn_out
```

Adds residual connection.

```
        # === MLP avec residu (pre-normalisation) ===
        normed = self.layer_norm_2(x)
        mlp_out = self.mlp(normed)
        x = x + mlp_out

        return x
```

Normalizes, applies MLP, adds residual, returns output.

Sentiment Transformer Class

```
class SentimentTransformer(nn.Module):
```

Transformer encoder with CLS token for sentiment classification.

```
    def __init__(self, vocab_size, embed_dim=256, num_heads=8,
                 mlp_dim=512, transformer_layers=4, max_length=60,
                 dropout=0.3, num_classes=3):
```

Initialization with parameters:

- `vocab_size`: Vocabulary size
- `embed_dim=256`: Embedding dimension
- `num_heads=8`: Attention heads
- `mlp_dim=512`: Feed-forward hidden size
- `transformer_layers=4`: Number of blocks
- `max_length=60`: Maximum sequence length
- `dropout=0.3`: Dropout rate
- `num_classes=3`: Output classes

```
        super().__init__()
        self.max_length = max_length
```

Stores maximum length.

```
        # Embedding de tokens + token CLS
        self.token_embedding = nn.Embedding(vocab_size, embed_dim,
            padding_idx=0)
```

Creates embedding layer with padding token at index 0.

```
        self.cls_token = nn.Parameter(torch.randn(1, 1, embed_dim))
```

Creates learnable CLS token prepended to sequences.

```
        # Positional embeddings appris (pour sequence + CLS token)
        self.pos_embed = nn.Parameter(torch.randn(1, max_length + 1,
            embed_dim))
```

Creates learnable positional embeddings for max_length + 1 positions (includes CLS).

```
        # Dropout regularisation
        self.dropout = nn.Dropout(dropout)
```

Creates dropout layer.

```
        # Couches Transformer
        self.transformer_blocks = nn.ModuleList([
            TransformerBlock(embed_dim, num_heads, mlp_dim, dropout)
            for _ in range(transformer_layers)
        ])
```

Creates list of 4 transformer blocks.

```
        # Tete de classification sur le token CLS
        self.mlp_head = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, num_classes)
        )
```

Creates classification head: normalization + linear ($256 \rightarrow 3$).

```
        # Initialisation des poids (Xavier pour stabilite)
        self._init_weights()
```

Calls weight initialization method.

Weight Initialization

```
    def _init_weights(self):
```

Custom initialization for training stability.

```
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.xavier_uniform_(m.weight)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
```

Initializes linear layers with Xavier uniform, biases with zeros.

```
            elif isinstance(m, nn.Embedding):
                nn.init.normal_(m.weight, std=0.02)
```

Initializes embeddings with small normal distribution.

```
            elif isinstance(m, nn.LayerNorm):
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
```

Initializes layer norms with ones (weight) and zeros (bias).

Forward Pass

```
    def forward(self, input_ids, attention_mask=None):
```

Forward pass with token indices and optional attention mask (1=real token, 0=padding).

```
        B = input_ids.size(0)
```

Gets batch size.

```
        # Embedding des tokens
        x = self.token_embedding(input_ids)  # [B, seq_len, embed_dim]
```

Converts indices to embeddings.

```
        # Ajout du token CLS en position 0
        cls_tokens = self.cls_token.expand(B, -1, -1)  # [B, 1, embed_dim]
        x = torch.cat((cls_tokens, x), dim=1)  # [B, seq_len+1, embed_dim]
```

Expands CLS token for batch and prepends to sequence.

```
        # Ajout des positional embeddings (tronquage si sequence trop longue
            )
        x = x + self.pos_embed[:, :x.size(1), :]
        x = self.dropout(x)
```

Adds positional embeddings (truncates if sequence exceeds max_length) and applies dropout.

```
        # CORRECTION CRITIQUE : key_padding_mask pour nn.MultiheadAttention
        # Format: [batch_size, seq_len+1] avec True = token a masquer (
            padding)
        key_padding_mask = None
        if attention_mask is not None:
```

CRITICAL CORRECTION: Prepares padding mask in correct format for `nn.MultiheadAttention`

```
            # Etendre le masque pour inclure le token CLS (toujours actif =
                False)
            cls_mask = torch.zeros(B, 1, device=attention_mask.device, dtype
                =torch.bool)
```

Creates mask for CLS token (False = don't mask it, always attend).

```
            padding_mask = (attention_mask == 0)  # True ou il y a du
                padding
```

Converts attention mask: True where there's padding (inverse of input mask).

```
            key_padding_mask = torch.cat((cls_mask, padding_mask), dim=1)  #
                [B, seq_len+1]
```

Concatenates CLS mask and padding mask. Final shape: (batch, seq_len+1).

```
        # Passage dans les couches Transformer
        for block in self.transformer_blocks:
            x = block(x, key_padding_mask=key_padding_mask)
```

Passes through all 4 transformer blocks with padding mask.

```
        # Classification sur le token CLS
        cls_output = x[:, 0]  # [B, embed_dim]
```

Extracts CLS token representation from position 0.

```
        logits = self.mlp_head(cls_output)  # [B, num_classes]

        return logits
```

Applies classification head and returns logits.

Tools Used

PyTorch Core (`torch`)

- `torch.randn`: Random initialization
- `torch.zeros`: Zero tensor creation
- `torch.cat`: Concatenates tensors
- `nn.Parameter`: Learnable parameters

Neural Network Module (`nn`)

- `nn.Module`: Base class
- `nn.Embedding`: Token embeddings with padding
- `nn.ModuleList`: Container for transformer blocks
- `nn.MultiheadAttention`: Built-in attention mechanism
- `nn.LayerNorm`: Layer normalization
- `nn.Linear`: Linear transformations
- `nn.GELU`: Activation function
- `nn.Dropout`: Regularization
- `nn.Sequential`: Chains layers

Initialization Methods

- `nn.init.xavier_uniform_`: Xavier initialization for linear layers
- `nn.init.normal_`: Normal initialization for embeddings
- `nn.init.zeros_`: Zero initialization for biases
- `nn.init.ones_`: One initialization for layer norm weights

Key Corrections and Features

- **key_padding_mask vs attn_mask**: Uses `key_padding_mask` for padding (correct format), not `attn_mask` (prevents dimension errors)
- **Mask format**: Boolean tensor where True = mask token, False = attend to token
- **CLS token masking**: Always False (never masked, always attends)
- **Padding mask inversion**: Converts input mask (1=real, 0=pad) to padding mask (True=pad, False=real)
- **Pre-normalization**: Normalizes before sublayers (more stable than post-norm)
- **Xavier initialization**: Improves convergence stability

- **GELU activation**: Better for transformers than ReLU
- **Learnable positional embeddings**: Trained during model training

Architecture Advantages

- **CLS token**: Single representation for entire sequence classification
- **Correct masking**: Prevents attention to padding tokens
- **Pre-LN**: More stable gradients during training
- **Multiple layers**: 4 transformer blocks for deeper representations
- **Proper initialization**: Xavier for weights, small normal for embeddings